



UNIVERSITI MALAYA

KOD KURSUS	:	WIX1002
NAMA KURSUS	:	FUNDAMENTALS OF PROGRAMMING
NAMA PENSYARAH	:	DR UNAIZAH HANUM BINTI OBAIDELLAH
SEMESTER & SESI	:	SEMESTER 1, 2024/2025
KUMPULAN	:	OCC 1

BIL.	NAMA	NO MATRIK
1	AHMAD MUKHLIS BIN MOHD NORDIN	23002476
2	AZNUR ANAK JITWIL @ JIKUIL	23001792
3	MUHAMMAD AFIQ BIN MOHD FAUZI	23002465
4	MUHAMMAD SYAMEER HARITH BIN MOHD YUSOF	23002196
5	JOECHELE LIM QIU YING	24004535

Contents

QUESTION 1	1
Problem.....	1
Solution	1
Sample Input and Output.....	2
Source code.....	3
QUESTION 2	8
Problem.....	8
Solution	9
Sample Input and Output.....	10
Source Code	13
QUESTION 3:	24
Problem Statement	24
Solution	25
Sample output.....	26
Source Code	26
QUESTION 4	31
Problem Statement	31
Solution	32
Sample input and output	34
Source Code.....	37
QUESTION 5	45
Problem.....	45
Solution	47
Sample Input and Output.....	50
Source Code.....	58
QUESTION 6	69
Problem.....	69
Solution	69
Sample input and output	71
Source code.....	72

QUESTION 1

Problem

In a bustling city where people rely on rented vehicles for their daily commutes, vacations, and business trips, a vehicle rental company offers a range of vehicles for customers to rent based on their needs. The company manages various vehicles such as cars, vans, and motorcycles with different rental rates. You are tasked with designing an advanced Vehicle class for a rental system that handles various types of vehicles, pricing models, and additional rental rules. The system must support fuel charges, and tiered discounting based on both vehicle type and rental duration.

Solution

A class Vehicle was created. The class will include the following item:

1. Constructor
 - a) An empty constructor that initializes:
 - i) vehicleID as "0000"
 - ii) type as "Car"
 - iii) brand as "Myvi"
 - iv) rentalRate as 100/day
 - v) fuelLevel as 100%
 - b) A constructor that accepts parameters for vehicleID, type, brand, rentalRate, and fuelLevel.
2. Accessor and mutator methods for all fields.
3. Methods
 - a) calculateRentalRate :The rental rate should also be adjusted based on vehicle type:
 - i) Vans: 20% more expensive than the base rate.
 - ii) Motorcycles: 25% less expensive than the base rate.
 - b) calculateRentalCost (int input) : A method to calculate the total cost of renting the vehicle based on discount tiers:
 - i) 5% discount for rentals between 7-14 days.
 - ii) 10% discount for rentals longer than 14 days.
 - c) calculateFuelCharge(): Calculates fuel charges based on the fuel level: - If fuel is below 50%, a RM50 penalty applies.
 - d) toString(): Override the method to return a detailed summary of the vehicle's rental information, including fuel level and the final total cost.

Sample Input and Output

```
run:
Vehicle ID: 0987
Type: Car
Brand: Mitsubishi
Rental Rate: RM100.00
Fuel Level: 50.00%
Rental Days : 7
Total Cost: RM95.00
Rental Cost for 10 days: RM95.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

```
run:
Vehicle ID: 0987
Type: van
Brand: Mitsubishi
Rental Rate: RM100.00
Fuel Level: 50.00%
Rental Days : 7
Total Cost: RM114.00
Rental Cost for 10 days: RM114.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

Source code

```
public class Q1 {

    public static void main(String[] args) {

        Vehicle vehicle1 = new Vehicle("0987", "van", "Mitsubishi", 100, 50);

        System.out.println(vehicle1);

        System.out.println("Rental Cost for 10 days: RM" + vehicle1.calculateRentalCost(10));

    }

}

class Vehicle{

    private String vehicleID;

    private String type;

    private String brand;

    private double rentalrate;

    private double fuellevel;

    //constructor without parameter

    public Vehicle(){

        vehicleID = "";

        type = "car";

        brand = "Myvi";

        rentalrate = 100;

        fuellevel = 100;

    }

}
```

```
//constructor with parameter

public Vehicle(String vehicleID, String type, String brand, double rentalrate, double
fuellevel){

    this.vehicleID = vehicleID;

    this.type = type;

    this.brand = brand;

    this.rentalrate = rentalrate;

    this.fuellevel = fuellevel;

}

//accessor for all var (getter)

public String getVehicleID(){

    return vehicleID;

}

public String getType(){

    return type;

}

public String getBrand(){

    return brand;

}

public double getRentalRate(){

    return rentalrate;

}

public double getFuelLevel(){
```

```

        return fuellevel;
    }

    //mutator for all var (setter)

    public void changeVehicleID(String vehicleID){

        this.vehicleID = vehicleID;
    }

    public void changeType(String type){

        this.type = type;
    }

    public void changeBrand(String brand){

        this.brand = brand;
    }

    public void changeRentalRate(double rentalrate){

        this.rentalrate = rentalrate;
    }

    public void changeFuelLevel(double fuellevel){

        this.fuellevel = fuellevel;
    }

    // calculate the rental rate here

    public double calculateRentalRate() {

        switch (type.toLowerCase()) {

```

```

    case "van":

        return rentalrate * 1.2; // 20% more for vans

    case "motorcycle":

        return rentalrate * 0.75; // 25% less for motorcycles

    default:

        return rentalrate; // normal rate

    }

}

```

//calculate cost if rent for long time

```

public double calculateRentalCost(int days){

    double discount;

    double baseCost = calculateRentalRate();

    if(days<7){

        discount = 1;

    }else if(days >= 7 && days<=14){

        discount = 0.95;

    }else{

        discount =0.9;

    }

}

```

```

double totalCost = baseCost * discount;

```

```

return totalCost;

```



```
}
```

```
//calculate fuel charges if less than 50, + 50
```

```
public double calculateFuelCharge() {
```

```
    if (fuellevel < 50) {
```

```
        return 50;
```

```
    }
```

```
    return 0;
```

```
}
```

```
// Override toString method to return vehicle information
```

```
@Override
```

```
public String toString() {
```

```
    double rentalCost = calculateRentalCost(10);
```

```
    double fuelCharge = calculateFuelCharge();
```

```
    double totalCost = rentalCost + fuelCharge;
```

```
    return String.format("Vehicle ID: %s\n"
```

```
        + "Type: %s\n"
```

```
        + "Brand: %s\n"
```

```
        + "Rental Rate: RM%.2f\n"
```

```
        + "Fuel Level: %.2f%%\n"
```

```
        + "Rental Days : 10\n"
```

```

        + "Total Cost: RM%.2f",
        vehicleID, type, brand, rentalrate, fuellevel, totalCost, fuelCharge);
    }
}

```

QUESTION 2

Problem

Genshin Impact is an exciting open-world game featuring the beautiful world of Teyvat, where selected individuals in the world are granted elemental powers in the form of accessories known as Visions. These special heroes utilize one of 4 elements, which are Pyro, Cryo, Hydro and Electro, to defeat monsters and enemies that threaten the peace of their world. However, each of these monsters have their own strengths and weaknesses against elements, which means some level of strategy is necessary. You are required to create a class Hero to store information about each hero, including their name, element and power. The element can only be one of the four aforementioned elements. You should also create a class Monster to store information about the monster, including their name, pyro resistance, hydro resistance, electro resistance, cryo resistance and their health points (HP). A class, HeroParty, is created to manage the heroes we have such that we may better organize and evaluate our heroes based on their combat abilities against monsters. First, sort and print the list of heroes based on their power. Then, we need to decide which heroes in the party can defeat a particular monster. Create a method battleWinners(Monster enemy) which will evaluate each hero's ability against the particular monster and print the name of heroes that will win. The hero is considered to win if their damage is equal to or exceeds the HP of the monster. The damage formula is given as:

$$\text{Damage} = \text{Power} \times (100\% - \text{ElementalRes}\%)$$

Note: ElementalRes refers to the corresponding elemental res based on the character's element. In addition to ordinary monsters, there may sometimes be boss monsters which are much stronger than their ordinary counterparts. Thankfully, heroes do not have to work alone and can get the support of 1 other hero to help them. The elements of both hero will interact to provide the main hero with a special buff. The buffs are as follows: can get the support of 1 other hero to

help them. The elements of both hero will interact to provide the main hero with a special buff. The buffs are as follows:

Element Combination	Buffs
Pyro x Hydro	Damage x1.5 for first hero
Pyro x Cryo	Damage x2.0 for first hero
Pyro x Electro	Extra damage=Random number between 50 to 100
Hydro x Electro	Extra damage=(Random number between 1-20)x5
Hydro x Cryo	-
Electro x Cryo	Reduce resistance for second hero by 10%

Note: Damage multiplier is calculated at the end, after deducting elemental resistance. Create a method called battleBoss(Monster boss), which calculates the pair of heroes that will deal the highest damage to the enemy and the value of the damage. Note that the order of heroes in the pair is important. Both heroes will deal damage to the boss based on the original equation, plus whatever relevant buffs added. The order of element combination does not matter. Print the pair of heroes with highest damage and the value of the damage.

[Solution](#)

The code begins with the declaration of the Hero class, where each hero is defined by their name, element, and power. The constructor initializes these attributes, ensuring the element is valid by checking against predefined values; otherwise, an error message is displayed. Next, the Monster class is declared, with attributes for name, elemental resistances (pyroRes, cryoRes, etc.), and health points (hp). The constructor validates the health value, assigning attributes only if it's positive, while also including a getName method to retrieve the monster's name. The HeroParty class manages an array of heroes, initialized through its constructor. The sortList method sorts heroes by power using a bubble sort algorithm, comparing and swapping adjacent elements iteratively. The getName method retrieves a hero's name by index. The battleWinners method calculates damage dealt by each hero to a monster based on their element and the monster's resistance, determining if the damage exceeds the monster's HP to declare a winner.

The battleBoss method handles pair-based battles against a boss monster. It uses nested if statements to evaluate synergies between hero elements, such as doubling damage for certain combinations or adding random bonus damage. Damage calculations take the hero's power, monster's resistance, and synergies into account, iterating through hero pairs to maximize damage. The method identifies the pair with the highest damage and prints their names along with the total damage dealt. The code integrates logical structures and randomness to simulate battles, incorporating validations, sorting, and conditional interactions for a dynamic gameplay experience.

Sample Input and Output

Example 1:

```
public static void main(String[] args) {
    Hero amber=new Hero("Amber", "Pyro", 80.0);
    Hero kaeya=new Hero("Kaeya", "Cryo", 60.0);
    Hero lisa=new Hero("Lisa", "Electro", 100.0);
    Hero barbara=new Hero("Barbara", "Hydro", 40.0);
    Hero[] herolist={amber, kaeya, lisa, barbara};
    HeroParty party=new HeroParty(herolist);
    party.sortList(herolist);
    System.out.println("List of heroes based on power : ");
    for(int i=0; i<herolist.length; i++){
        System.out.println(party.getName(i));
    }
    Monster mitachurl=new Monster("Mitachurl", 10, 10, 10, 10, 80.0);
    System.out.println("\nHeroes that will win against "+mitachurl.getName()+" : ");
    party.battleWinners(mitachurl);
    Monster cryoRegisvine=new Monster("Cryo Regisvine", 20, 40, 20, 20, 200.0);
    party.battleBoss(cryoRegisvine);
}
```

List of heroes based on power :

Barbara

Kaeya

Amber

Lisa

Heroes that will win against Mitachurl :

Lisa

The pair with the highest damage (not in order) : Lisa and Amber

Total damage dealt : 228.0

Example 2:

```
public static void main(String[] args) {
    Hero xiangling=new Hero("Xiangling","Pyro",120.0);
    Hero ganyu=new Hero("Ganyu","Cryo",180.0);
    Hero beidou=new Hero("Beidou","Electro",80.0);
    Hero xingqiu=new Hero("Xingqiu","Hydro",100.0);
    Hero[]herolist={xiangling,ganyu,beidou,xingqiu};
    HeroParty party=new HeroParty(herolist);
    party.sortList(herolist);
    System.out.println("List of heroes based on power : ");
    for(int i=0;i<herolist.length;i++){
        System.out.println(party.getName(i));
    }
    Monster fatuiAgent=new Monster("Fatui Agent",10,10,10,10,150.0);
    System.out.println("\nHeroes that will win against "+fatuiAgent.getName()+" : ");
    party.battleWinners(fatuiAgent);
    Monster primoGeovishap=new Monster("Primo Geovishap",20,20,20,50,200.0);
    party.battleBoss(primoGeovishap);
}
```

List of heroes based on power :

Beidou

Xiangling

Xingqiu

Ganyu

Heroes that will win against Fatui Agent :

Ganyu

The pair with the highest damage (not in order) : Xingqiu and Xiangling

Total damage dealt : 224.0

Example3:

```
public static void main(String[] args) {  
    Hero arlecchino=new Hero("Arlecchino","Pyro",280.0);  
    Hero wriothesley=new Hero("Wriothesley","Cryo",180.0);  
    Hero clorinde=new Hero("Clorinde","Electro",220.0);  
    Hero neuvillette=new Hero("Neuvillette","Hydro",300.0);  
    Hero[]herolist={arlecchino,wriothesley,clorinde,neuvillette};  
    HeroParty party=new HeroParty(herolist);  
    party.sortList(herolist);  
    System.out.println("List of heroes based on power : ");  
    for(int i=0;i<herolist.length;i++){  
        System.out.println(party.getName(i));  
    }  
    Monster clockworkMeka=new Monster("Clockwork Meka",20,20,20,20,200.0);  
    System.out.println("\nHeroes that will win against "+clockworkMeka.getName()+" : ");  
    party.battleWinners(clockworkMeka);  
    Monster pyroRegisvine=new Monster("Pyro Regisvine",80,20,0,20,300.0);  
    party.battleBoss(pyroRegisvine);  
}
```

List of heroes based on power :

Clorinde

Wriothesley

Arlecchino

Neuvillette

Heroes that will win against Clockwork Meka :

Arlecchino

Neuvillette

The pair with the highest damage (not in order) : Neuvillette and Arlecchino

Total damage dealt : 506.0

Source Code

```
import java.util.Random;

public class Viva3Q2 {

    public static class Hero{

        public String name=null;

        public String element=null;

        public double power=0.00;

        public Hero(String name, String element, double power){

            if(element.equals("Pyro")||element.equals("Cryo")||element.equals("Hydro")||element.equals("Ele
            ctro")){

                this.name=name;

                this.element=element;

                this.power=power;

            }

            else{

                System.out.println("Invalid element input!!");

            }

        }

    }

    public static class Monster{

        public String name=null;

        public int pyroRes=0;

        public int cryoRes=0;
```

```

    public int hydroRes=0;

    public int electroRes=0;

    public double hp=0.0;

    public Monster(String name,int pyroRes,int cryoRes,int hydroRes,int electroRes,double
hp){

        if(hp>0.0){

            this.name=name;

            this.pyroRes=pyroRes;

            this.cryoRes=cryoRes;

            this.hydroRes=hydroRes;

            this.electroRes=electroRes;

            this.hp=hp;

        }

        else{

            System.out.println("Invalid HP input!!");

        }

    }

    public String getName(){

        return name;

    }

}

public static class HeroParty{

    Random rd=new Random();

    public Hero []party;

```



```

public HeroParty(Hero []party){

    this.party=party;

}

public void sortList(Hero[]h){

    int length=h.length;

    for(int i=0;i<h.length-1;i++){

        for(int j=0;j<length-1;j++){

            if(h[i].power>h[j+1].power){

                Hero temp=h[j];

                h[j]=h[j+1];

                h[j+1]=temp;

            }

        }

        length--;

    }

}

public String getName(int i){

    return party[i].name;

}

double damage;

public void battleWinners(Monster x){

    for(int i=0;i<party.length;i++){

        switch (party[i].element){

```

```

        case "Pyro":
            damage=(party[i].power)*((100.0-(x.pyroRes))/100.0);
            break;

        case "Cryo":
            damage=(party[i].power)*((100.0-(x.cryoRes))/100.0);
            break;

        case "Hydro":
            damage=(party[i].power)*((100.0-(x.hydroRes))/100.0);
            break;

        case "Electro":
            damage=(party[i].power)*((100.0-(x.electroRes))/100.0);
            break;
    }

    if(damage>=x.hp){
        System.out.println(party[i].name);
    }
}

}

public void battleBoss(Monster m){
    int ch=50+rd.nextInt(51);

    int ext=1+rd.nextInt(20); // Random number from 0 to 19

    double min;

    damage=0.0;

```

```

int i=0;

do{

    double prev;

    min=damage;

    if((party[i].element.equals("Pyro")&&party[i+1].element.equals("Cryo"))||(party[i].element.equals("Cryo")&&party[i+1].element.equals("Pyro"))){

        if(party[i].element.equals("Pyro")){

            damage=(party[i].power)*((100.0-(m.pyroRes))/100);

            damage=2.00*damage;    //doubles damage for first hero

            damage=damage+((party[i+1].power)*((100.0-(m.cryoRes))/100.0));

            prev=damage;

            damage=(party[i+1].power)*((100.0-(m.cryoRes))/100.0);

            damage=2.00*damage;    //doubles damage for first hero

            damage=damage+((party[i].power)*((100.0-(m.pyroRes))/100.0));

            if(prev>=damage){

                damage=prev;

            }

        }

    }

    else if(party[i].element.equals("Cryo")){

        damage=(party[i].power)*((100.0-(m.cryoRes))/100.0);

        damage=2.00*damage;    //doubles damage for first hero

        damage=damage+((party[i+1].power)*((100.0-(m.pyroRes))/100.0));

        prev=damage;
    }
}

```

```

        damage=(party[i+1].power)*((100.0-(m.pyroRes))/100.0);

        damage=2.00*damage;    //doubles damage for first hero

        damage=damage+((party[i].power)*((100.0-(m.cryoRes))/100.0));

        if(prev>=damage){

            damage=prev;

        }

    }

}

else

if((party[i].element.equals("Pyro")&&party[i+1].element.equals("Hydro"))||(party[i].element.equals("Hydro")&&party[i+1].element.equals("Pyro"))){

    if(party[i].element.equals("Pyro")){

        damage=(party[i].power)*((100.0-(m.pyroRes))/100.0);

        damage=1.50*damage;    //multiplies first hero's damage by 1.5

        damage=damage+((party[i+1].power)*((100.0-(m.hydroRes))/100.0));

        prev=damage;

        damage=(party[i+1].power)*((100.0-(m.hydroRes))/100.0);

        damage=1.50*damage;    //multiplies first hero's damage by 1.5

        damage=damage+((party[i].power)*((100.0-(m.pyroRes))/100.0));

        if(prev>=damage){

            damage=prev;

        }

    }

}

else if(party[i].element.equals("Hydro")){

```

```

        damage=(party[i].power)*((100.0-(m.hydroRes))/100.0);

        damage=1.50*damage;    //multiplies first hero's damage by 1.5

        damage=damage+((party[i+1].power)*((100.0-(m.pyroRes))/100.0));

        prev=damage;

        damage=(party[i+1].power)*((100.0-(m.pyroRes))/100.0);

        damage=1.50*damage;    //multiplies first hero's damage by 1.5

        damage=damage+((party[i].power)*((100.0-(m.hydroRes))/100.0));

        if(prev>=damage){

            damage=prev;

        }

    }

}

else

if((party[i].element.equals("Pyro")&&party[i+1].element.equals("Electro"))||(party[i].element.equals("Electro")&&party[i+1].element.equals("Pyro"))){

    if(party[i].element.equals("Pyro")){

        damage=(party[i].power)*((100.0-(m.pyroRes))/100.0);

        damage=damage+((party[i+1].power)*((100.0-(m.electroRes))/100.0));

        prev=damage;

        damage=(party[i+1].power)*((100.0-(m.electroRes))/100.0);

        damage=damage+((party[i].power)*((100.0-(m.pyroRes))/100.0));

        if(prev>=damage){

            damage=prev;

        }

    }

}

```

```

    }

    else if(party[i].element.equals("Electro")){

        damage=(party[i].power)*((100.0-(m.electroRes))/100.0);

        damage=damage+((party[i+1].power)*((100.0-(m.pyroRes))/100.0));

        prev=damage;

        damage=(party[i+1].power)*((100.0-(m.pyroRes))/100.0);

        damage=damage+((party[i].power)*((100.0-(m.electroRes))/100.0));

        if(prev>=damage){

            damage=prev;

        }

    }

    damage=damage+ch;    //extra damage (random number between 50 - 100)

}

else

if((party[i].element.equals("Electro")&&party[i+1].element.equals("Hydro"))||(party[i].element.e
quals("Hydro")&&party[i+1].element.equals("Electro"))){

    if(party[i].element.equals("Electro")){

        damage=(party[i].power)*((100.0-(m.electroRes))/100.0);

        damage=damage+((party[i+1].power)*((100.0-(m.hydroRes))/100.0));

        prev=damage;

        damage=(party[i+1].power)*((100.0-(m.hydroRes))/100.0);

        damage=damage+((party[i].power)*((100.0-(m.electroRes))/100.0));

        if(prev>=damage){

            damage=prev;

```

```

    }
}
else if(party[i].element.equals("Hydro")){

    damage=(party[i].power)*((100.0-(m.hydroRes))/100.0);

    damage=damage+((party[i+1].power)*((100.0-(m.electroRes))/100.0));

    prev=damage;

    damage=(party[i+1].power)*((100.0-(m.electroRes))/100.0);

    damage=damage+((party[i].power)*((100.0-(m.hydroRes))/100.0));

    if(prev>=damage){

        damage=prev;

    }

}

    damage=damage+(5.0*ext); //extra damage (random number between 1-20
multiplied by 5)

}

else

if((party[i].element.equals("Electro")&&party[i+1].element.equals("Cryo"))||(party[i].element.equals("Cryo")&&party[i+1].element.equals("Electro"))){

    if(party[i].element.equals("Electro")){

        damage=(party[i].power)*((100.0-(m.electroRes))/100.0);

        damage=damage+((party[i+1].power)*((100.0-(m.cryoRes-10.0))/100.0));

//reduces the enemy's resistance based on second hero's element

        prev=damage;

        damage=(party[i+1].power)*((100.0-(m.cryoRes))/100);

```

```

        damage=damage+((party[i].power)*((100.0-(m.electroRes-10.0))/100.0));

        if(prev>=damage){

            damage=prev;

        }

    }

    else if(party[i].element.equals("Cryo")){

        damage=(party[i].power)*((100.0-(m.cryoRes))/100);

        damage=damage+((party[i+1].power)*((100.0-(m.electroRes-10.0))/100.0));

//reduces the enemy's resistance based on second hero's element

        prev=damage;

        damage=(party[i+1].power)*((100.0-(m.electroRes))/100);

        damage=2.00*damage;    //doubles damage for first hero

        damage=damage+((party[i].power)*((100.0-(m.cryoRes-10.0))/100.0));

        if(prev>=damage){

            damage=prev;

        }

    }

}

else

if((party[i].element.equals("Hydro")&&party[i+1].element.equals("Cryo"))||(party[i].element.equals("Cryo")&&party[i+1].element.equals("Hydro"))){

    if(party[i].element.equals("Hydro")){

        damage=(party[i].power)*((100.0-(m.hydroRes))/100.0);

```



```
        damage=damage+((party[i+1].power)*((100.0-(m.cryoRes))/100.0));    //total  
damage of both heroes
```

```
    prev=damage;
```

```
    damage=(party[i+1].power)*((100.0-(m.cryoRes))/100.0);
```

```
    damage=damage+((party[i].power)*((100.0-(m.hydroRes))/100.0));
```

```
    if(prev>=damage){
```

```
        damage=prev;
```

```
    }
```

```
}
```

```
else if(party[i].element.equals("Cryo")){
```

```
    damage=(party[i].power)*((100.0-(m.cryoRes))/100.0);
```

```
    damage=damage+((party[i+1].power)*((100.0-(m.hydroRes))/100.0));    //total
```

```
damage of both heroes
```

```
    prev=damage;
```

```
    damage=(party[i+1].power)*((100.0-(m.hydroRes))/100.0);
```

```
    damage=damage+((party[i].power)*((100.0-(m.cryoRes))/100.0));
```

```
    if(prev>=damage){
```

```
        damage=prev;
```

```
    }
```

```
}
```

```
}
```

```
i++;
```

```
if(i+1>=party.length){
```

```
    break;
```

```

    }

    }while(damage>min);

    System.out.print("The pair with the highest damage (not in order) : ");

    if(i>=1){

        System.out.println(party[i].name+" and "+party[i-1].name);

    }

    System.out.println("Total damage dealt : "+damage);

}

}

```

QUESTION 3:

Problem Statement

Write a Java program where you are required to implement a class called Room with the following specifications:

Instance Variables:

- private String roomNumber: Unique room identifier (e.g., "101").
- private String roomType: Room type ("Single", "Double", or "Suite").
- private double rentalPrice: Rental price per night.

Static Variable:

- private static int totalRooms: Counts the total number of rooms created.

Constructor:

- Accepts roomNumber, roomType, and rentalPrice as parameters, initializes instance variables, and increments totalRooms.

Encapsulation:

- Provide public getter and setter methods for each instance variable.

Method Overloading:

- calculateRent(int days): Calculates total rent for the specified number of days.
- calculateRent(int days, double discount): Calculates total rent after applying a discount.

Wrapper Class:

- getTotalRooms(): Returns totalRooms as an Integer object.

Static Method:

- displayRoomDetails(Room room): Prints room details, including room number, type, rental price, and total rooms.

In the main method:

- Instantiate at least **Two (2)** Room objects.
- Use the methods to display room details and calculate rent with and without a discount.

Solution

First, create a class Room and declare 3 instance variables, 2 variables with String object named roomNumber and roomType, 1 variable with data type double called rentalPrice. Next, declare an integer static variable named totalRooms to 0 as initial value. Also declare all these variables with private access modifier to restrict its access from other classes. Then, create a constructor that accepts all 3 instance variables (roomNumber, roomType, rentalPrice) as parameters to initialize these variables and also increments totalRooms. After that, create mutator methods for all instance variables to set value to the variable itself. Also create an accessor method for all instance variables to return value of variable when the object wants to access it. Next, create a method that accepts number of days for calculate rental price according to number of days given. In this method, declare a variable named price with data type double. Multiply rentalPrice with days given and assign this equation to variable price. Then, print the number of days and total rent. Next, overload this method with additional parameter that accepts discount price. Same as before, declare a variable named discprice with data type double. Multiply rentalPrice with days given and minus discount given and assign this equation to variable discprice. Print the number of days, discount given and total rent after discount. After that, create a wrapper class Integer static method that

return number of totalRooms created. Lastly, in this class Room, create a method called displayRoomDetails that accepts room object as parameter. In this method, display room number by getting the value from accessor method. Repeat this step to display room type and rental price also print value of totalRooms.

In the main class, declare Room object called room1 and assign the value of roomNumber, roomType, and rentalPrice to the object. Call method displayRoomDetails from class Room and assign room1 as the parameter. After that, calculate rent for room1 by calling calculateRent method and assign number of days to stay. Calling the same method but with a discount price to calculate rental price for room1 after discount. Repeat all step for new object called room2 with different value of roomNumber, roomType, rentalPrice, number of days to stay and discount price.

Sample output

```
Room Number: 101
Room Type: Single
Rental Price: $100.0
Total Rooms: 1
Total Rent for 5 days: $500.0
Total Rent for 5 with $50.0 discount: $450.0
```

```
Room Number: 106
Room Type: Double
Rental Price: $167.0
Total Rooms: 2
Total Rent for 3 days: $501.0
Total Rent for 3 with $55.0 discount: $446.0
```

```
-----
BUILD SUCCESS
-----
```

Source Code

Room Class

```
public class Room {

    private String roomNumber;

    private String roomType; //(Single,Double,Suite)

    private double rentalPrice; //per night
```

```
private static int totalRooms = 0; //total rooms created
```

```
public Room(String roomNumber, String roomType, double rentalPrice){
```

```
    this.roomNumber = roomNumber;
```

```
    this.roomType = roomType;
```

```
    this.rentalPrice = rentalPrice;
```

```
    totalRooms++;
```

```
}
```

```
public void setRoomNumber(String roomNumber){
```

```
    this.roomNumber = roomNumber;
```

```
}
```

```
public void setRoomType(String roomType){
```

```
    this.roomType = roomType;
```

```
}
```

```
public void setRentalPrice(double rentalPrice){
```

```
    this.rentalPrice = rentalPrice;
```

```
}
```

```
public String getRoomNumber(){
```

```
    return roomNumber;
```

```
}
```

```
public String getRoomType(){
```

```
    return roomType;
```

```
}
```

```
public double getRentalPrice(){
```

```
    return rentalPrice;
```

```
}
```

```
//Method overloading
```

```
public void calculateRent(int days){
```

```
    double price = rentalPrice*days;
```

```
    System.out.println("Total Rent for " + days + " days: $" + price);
```

```
}
```

```

public void calculateRent(int days, double discount){

    double discprice = rentalPrice*days - discount;

    System.out.println("Total Rent for " + days + " with $" + discount + " discount: $" +
discprice);

}

//Wrapper Class

public static Integer getTotalRooms(){

    return totalRooms;

}

public static void displayRoomDetails(Room room){

    System.out.println("Room Number: " + room.getRoomNumber());

    System.out.println("Room Type: " + room.getRoomType());

    System.out.println("Rental Price: $" + room.getRentalPrice());

    System.out.println("Total Rooms: " + getTotalRooms());

}

}

```

Main Class

```
public class Viva3 {  
    public static void main(String[] args) {  
  
        Room room1 = new Room("101","Single",100);  
        Room.displayRoomDetails(room1);  
        room1.calculateRent(5);  
        room1.calculateRent(5, 50);  
        System.out.println();  
  
        Room room2 = new Room("106","Double",167);  
        Room.displayRoomDetails(room2);  
        room2.calculateRent(3);  
        room2.calculateRent(3, 55);  
        System.out.println();  
  
    }  
}
```


QUESTION 4

Problem Statement

Create a simplified text-based adventure game in Java. The game has four primary entities: Player, Enemy, Item and Black Magic. Implement the following classes to manage these entities and create basic game interactions.

1. Player Class:

- Attributes:
- String name: The name of the player.
- int health: The player's health points (default is 100).
- int attackPower: The power of the player's attacks (default is 10).
- Methods:
- Constructor that takes the player's name and initializes health to 100 and attackPower to 10.
- int attack(Enemy enemy): Reduces the enemy's health based on the player's attackPower.
- Accessor and mutator for attackPower.
- boolean isAlive(): Returns true if the player's health is greater than 0.
- Include a toString method that returns the combat log

2. Enemy Class:

- Attributes:
- String type: The type of enemy (e.g., "Goblin", "Orc").
- int health: The enemy's health points (default is 50).
- int attackPower: The power of the enemy's attacks (default is 5).
- Methods:
- Constructor that takes the type of the enemy and initializes health to 50 and attackPower to 5.
- void attack(Player player): Reduces the player's health based on the enemy's attackPower.
- boolean isAlive(): Returns true if the enemy's health is greater than 0.
- Include a toString method that returns the combat log.

3. Item Class:

- Attributes:
- String name: The name of the item (but not limited to)
- Health Potion o Sword
- int effect: The effect the item has
- Health Potion: Increases the Player's health by 20 o Sword: Increases the Player's next attack power by 15. o Methods:
- Constructor that initializes the item's name and effect.
- int use(Player player): Uses the item's effect o Include a toString method that returns the combat log.

4. Black Magic Class: (used by Enemy)

- Attributes:
- String name: The name of the magic (but not limited to:)
- Poison o Magic Orb
- int effect: The effect the item has
- Poison: Damages the Player by 10, reducing the Player's next attack power by 10%
- Magic Orb: Increases the Enemy's next attack power by 200% (300% of current attack power) o Methods:
- Constructor that initializes the item's name and effect.
- int use(Player player, Enemy enemy): Uses the BlackMagic to enhance attack power or damage the player
- Include a toString method that returns the combat log.

Solution

- Firstly, create a player class. It represents the Hero controlled by the player. In the class, declare name, health which is initialized to 100 and attackPower which is initialized to 10. Create a mutator and accessor method for attackPower. Create a isAlive method that checks if the player's health is above 0. Then, create an attack method that reduces the

enemy's health based on the player's attack power. Lastly, create a toString method to return a combat log.

- Secondly, create an enemy class. It represents the Goblin controlled by the Enemy. In the class, declare type, health which is initialized to 50 and attackPower which is initialized to 5. Create a isAlive method that checks if the enemy's health is above 0. Then, create an attack method that reduces the player's health based on the enemy's attack power. Lastly, create a toString method to return a combat log.
- After that, create an Item class. It Represents items that the player can use to gain advantages. In the class declare name and effect. Items have a name such as "Health Potion" or "Sword" and an effect. The use() method applies the item's effect to the player. Health Potion Increases the player's health while Sword Increases the player's attack power. Then create a toString method to return the usage of the item log.
- Lastly, create a BlackMagic class. Represents magical effects used by the enemy. In the class declare name and effect. BlackMagic have a name such as "Health Poison" and "Magic Orb" and an effect. The use() method applies the BlackMagic's effect to the player and/or enemy. Poison Reduces the player's health and decreases the player's attack power by 10% while Magic Orb Triples the enemy's attack power. Then create a toString method to return the usage of the BlackMagic log.
- Then, in the main class Create a Player object with "Hero". Create an Enemy object with a "Goblin". Create an Item objects like a "Health Potion" that restores health and a "Sword" that increases attack power. Create a BlackMagic objects like a "Poison" that reduces the player's health and attackPower and a "Magic Orb" that increases the enemy attack power. The game continues in a loop while both the hero and goblin are alive (player.isAlive() and enemy.isAlive()). The player and enemy take turns attacking each other using the attack methods. (written in combat log that will be generated by toString method) .The player is prompted to choose an action either to Attack the goblin or Use an item (Health Potion or Sword). The chosen action is executed, affecting either the goblin or the player's stats. After that, The goblin chooses an action either to Attack the hero or Use black magic (Poison or Magic Orb). The chosen action is executed, affecting either the hero or the goblin's stats. When either the hero or goblin's health drops to 0. If the hero defeats the goblin, a victory message is displayed. If the goblin defeats the hero, a defeat message is displayed.

Sample input and output

```
*****
*           Hero vs Goblin Adventure!           *
*****
```

Hero's turn:

1.Attack

2.Use item

Move:

1

Hero attacked Goblin with attack power 10!

Goblin's turn:

1.Attack

2.Use black magic

Move:

2

Black Magic:

1.poison(-10 hp and -10% attack power player)

2.Magic orb(+200% attack power enemy)

Move:

1

Poison has been used! -10 to hero's hp and -10% to the hero's attack power for the next attack

Hero's turn:

1.Attack

2.Use item

Move:

1

Hero attacked Goblin with attack power 9!

```
Goblin's turn:
1.Attack
2.Use black magic
Move:
2
Black Magic:
1.poison(-10 hp and -10% attack power player)
2.Magic orb(+200% attack power enemy)
Move:
2
Magic Orb has been used! Goblin's attack power is now 200% stronger!
```

```
Hero's turn:
1.Attack
2.Use item
Move:
1
Hero attacked Goblin with attack power 9!
```

```
Goblin's turn:
1.Attack
2.Use black magic
Move:
1
Goblin attacked Hero with attack power 15!
```

Hero's turn:

1.Attack

2.Use item

Move:

2

Item:

1.Health potion(+20 hp)

2.Sword(+15 attack power)

Move:

2

Sword has been used! +15 attack power to hero!

Goblin's turn:

1.Attack

2.Use black magic

Move:

1

Goblin attacked Hero with attack power 15!

Hero's turn:

1.Attack

2.Use item

Move:

2

Item:

1.Health potion(+20 hp)

2.Sword(+15 attack power)

Move:

1

Health potion has been used! +20 health to hero!

```
Goblin's turn:
1.Attack
2.Use black magic
Move:
2
Black Magic:
1.poison(-10 hp and -10% attack power player)
2.Magic orb(+200% attack power enemy)
Move:
2
Magic Orb has been used! Goblin's attack power is now 200% stronger!
```

```
Hero's turn:
1.Attack
2.Use item
Move:
1
Hero attacked Goblin with attack power 24!
Goblin has been defeated by Hero! Hero wins!
BUILD SUCCESSFUL (total time: 1 minute 12 seconds)
```

[Source Code](#)

```
package test;

import java.util.Scanner;

class Player {

    public String name;
    public int health;
    public int attackPower;

    public Player(String name) {
        this.name = name;
        this.health = 100;
    }
}
```

```

        this.attackPower = 10;
    }

    public int getAttackPower() {
        return attackPower;
    }

    public void setAttackPower(int attackPower) {
        this.attackPower = attackPower;
    }

    public boolean isAlive() {
        return this.health > 0;
    }

    public int attack(Enemy enemy) {
        if (isAlive()) {
            enemy.health -= attackPower;
            System.out.println(toString());
            return attackPower; //
        } else {
            return 0;
        }
    }

    @Override
    public String toString() {
        return name + " attacked Goblin with attack power " + attackPower + "!";
    }

}

```



```

class Enemy {

    public String type;
    public int health;
    public int attackPower;

    public Enemy(String type) {
        this.type = type;
        this.health = 50;
        this.attackPower = 5;
    }

    public boolean isAlive() {
        return this.health > 0;
    }

    public void attack(Player player) {
        if (isAlive()) {
            player.health -= attackPower;
            System.out.println(toString());
        }
    }

    @Override
    public String toString() {
        return type + " attacked Hero with attack power " + attackPower + "!";
    }

}

class Item {

```

```

private String name;
private int effect;

public Item(String name, int effect) {
    this.name = name;
    this.effect = effect;
}

public int use(Player player) {
    if (name.equalsIgnoreCase("Health Potion")) {
        player.health += effect;
    } else if (name.equalsIgnoreCase("Sword")) {
        player.attackPower += effect;
    }
    System.out.println(toString());
    return effect;
}

@Override
public String toString() {
    if (name.equalsIgnoreCase("Health Potion")) {
        return name + " has been used! +" + effect + " health to hero!";
    } else if (name.equalsIgnoreCase("Sword")) {
        return name + " has been used! +" + effect + " attack power to hero!";
    } else {
        return "That item is not available!";
    }
}

}

class BlackMagic {

```

```

private String name;
private int effect;

public BlackMagic(String name, int effect) {
    this.name = name;
    this.effect = effect;
}

public int use(Player player, Enemy enemy) {
    if (name.equalsIgnoreCase("Poison")) {
        player.health -= effect;
        player.attackPower = ((int) (player.attackPower * 0.9));
    } else if (name.equalsIgnoreCase("Magic Orb")) {
        enemy.attackPower *= effect;
    }
    System.out.println(toString());
    return effect;
}

@Override
public String toString() {
    if (name.equalsIgnoreCase("Poison")) {
        return name + " has been used! -" + effect + " to hero's hp and -10% to the hero's attack
power for the next attack";
    } else if (name.equalsIgnoreCase("Magic Orb")) {
        return name + " has been used! Goblin's attack power is now 200% stronger!";
    } else {
        return "That item is not available!";
    }
}
}

```

```

public class Test {

public static void main(String[] args) {
    Scanner s = new Scanner(System.in);
    Player player = new Player("Hero");
    Enemy enemy = new Enemy("Goblin");
    Item healthpotion = new Item("Health potion", 20);
    Item sword = new Item("Sword", 15);
    BlackMagic poison = new BlackMagic("Poison", 10);
    BlackMagic magicorb = new BlackMagic("Magic Orb", 3);
    int move, move2;
    System.out.println("*****");
    System.out.println("*      Hero vs Goblin Adventure!      *");
    System.out.println("*****");
    while (player.isAlive() && enemy.isAlive()) {
        System.out.println("Hero's turn:\n1.Attack\n2.Use item\nMove: ");
        move = s.nextInt();
        switch (move) {
            case 1:
                player.attack(enemy);
                break;
            case 2:
                System.out.println("Item:\n1.Health potion(+20 hp)\n2.Sword(+15 attack
power)\nMove: ");
                move2 = s.nextInt();
                switch (move2) {
                    case 1:
                        healthpotion.use(player);
                        break;
                    case 2:
                        sword.use(player);
                        break;
                    default:

```

```

        System.out.println("No move!");
        break;
    }
    break;
default:
    System.out.println("No move!");
    break;
}
if (!enemy.isAlive()) {
    System.out.println(enemy.type + " has been defeated by " + player.name + "! " +
player.name + " wins!");
    System.exit(0);
}

System.out.println("Goblin's turn:\n1.Attack\n2.Use black magic\nMove: ");
move = s.nextInt();
switch (move) {
    case 1:
        enemy.attack(player);
        break;
    case 2:
        System.out.println("Black Magic:\n1.Poison(-10 hp and -10% attack power
player)\n2.Magic orb(+200% attack power enemy)\nMove: ");
        move2 = s.nextInt();
        switch (move2) {
            case 1:
                poison.use(player, enemy);
                break;
            case 2:
                magicorb.use(player, enemy);
                break;
            default:

```

```

        System.out.println("No move!");
        break;
    }
    break;
default:
    System.out.println("No move!");
    break;
}
if (!player.isAlive()) {
    System.out.println(player.name + " has been defeated by " + enemy.type + "! " +
enemy.type + " wins!");
    System.exit(0);
}
}

}

```

QUESTION 5

Problem

In a world where stories come alive through cinema, you are tasked with creating a movie collection system that allows users to manage their movie collection seamlessly. This system should enable users to easily add, remove, search, and display their movies based on various criteria.

Your mission is to create a Java application that manages a collection of movies. The application should include the following components:

1. **Movie Class:**

- **Attributes:**
 - *title*
 - *director*
 - *year*
 - *type*
- **Constructor:** A constructor that initializes all attributes.
- **Accessors:** Methods to retrieve the *title*, *director*, *year*, and *type* of the movie.
- **toString() Method:** Override the **toString** method to return a formatted string representation of the movie details.

2. **MovieCollection Class:**

- **Attributes:**
 - *movieCollection*
 - *numMovie*
- **Constructor:** Initializes the movie collection with a maximum capacity (e.g., 100).
- **Methods:**
 - **isEmpty():** Returns true if the collection is empty, otherwise false.
 - **addMovie(Movie movie):** Adds a movie to the collection. If the collection is full, it should print a message.
 - **removeMovie(String title):** Removes a movie by its title. If the movie is not found, it should print an appropriate message.

- **findmovieCollectionByDirector(String director):** Prints all movies directed by the specified director. If none are found, it should print a message.
- **findMovieByTitle(String title):** Searches for a movie by its title and prints its details if found.
- **sortmovieCollectionByReleaseYear():** Sorts the movie collection by release year in ascending order.
- **findmovieCollectionByReleaseYearRange(int startYear, int endYear):** Prints all movies released within the specified year range.
- **displayCollection():** Displays all movies in the collection.

Test Program

```
public class TestProgram {

    public static void main(String[] args) {
        MovieCollection movieCollection = new MovieCollection();

        Movie movie1 = new Movie("Inception", "Christopher Nolan", 2010, "Science
                                   Fiction");
        Movie movie2 = new Movie("The Shawshank Redemption", "Frank Darabont",
                                   1994, "Drama");
        Movie movie3 = new Movie("Pulp Fiction", "Quentin Tarantino", 1994,
                                   "Crime");
        Movie movie4 = new Movie("The Dark Knight", "Christopher Nolan", 2008,
                                   "Action");
        Movie movie5 = new Movie("The Godfather", "Francis Ford Coppola", 1972,
                                   "Crime");

        movieCollection.addMovie(movie1);
        movieCollection.addMovie(movie2);
        movieCollection.addMovie(movie3);
        movieCollection.addMovie(movie4);
    }
}
```



```
movieCollection.addMovie(movie5);

System.out.println("Searching for movie(s) by director:");
    movieCollection.findmovieCollectionByDirector("Christopher Nolan");

System.out.println("\nSearching for a movie by title:");
    movieCollection.findMovieByTitle("Inception");

System.out.println("\nMovies released between 1990 and 2010:");
movieCollection.findmovieCollectionByReleaseYearRange(1990, 2010);
System.out.println("\nSorted movieCollection by release year:");
movieCollection.sortmovieCollectionByReleaseYear();
movieCollection.displayCollection();

movieCollection.removeMovie(movie3.getTitle());
System.out.println("\nUpdated Movie Collection:");
movieCollection.displayCollection();
}
}
```

Solution

1. A class **Movie** is created to store movie information. A constructor for the **Movie** class is defined with parameters for the title, director, type, and year. Accessor and mutator methods are implemented for each variable to retrieve and modify their values. A **toString** method is created to return a formatted string describing the movie's details.
2. A class **MovieCollection** is created to store an array of **Movie** objects, named `movieCollection`. A constructor initializes the `movieCollection` array and a variable, `numMovie`, which keeps track of the number of movies in the collection. The maximum number of movies that can be stored is set to 100.
 - A boolean method, **isEmpty**, checks whether the collection contains any movies.
 - The **addMovie** method adds a movie to the array. The `numMovie` variable serves as the index for the next movie in the array. If the array is full, a message is printed indicating that no more movies can be added.

- The **removeMovie** method removes a movie from the collection. It retrieves the movie's title using the getTitle accessor method from the *Movie* class and compares it to the titles of movies in the array. If a match is found, the movie is deleted, and the remaining movies are shifted forward to fill the gap. A message is printed to confirm the deletion. If no such movie is found, a message is printed to indicate this.
 - The **findMovieCollectionByDirector** method searches for all movies directed by a specific director. A boolean variable, found, tracks whether any matches are found. If movies by the director are found, their information is printed; otherwise, a "No movie found!" message is displayed.
 - The **findMovieByTitle** method searches for a movie by its title. Similar to the previous method, a boolean variable, found, is used to track whether a match is found. If a movie with the given title is located, its information is printed; otherwise, a "No movie found!" message is displayed.
 - The **sortMovieCollectionByReleaseYear** method sorts the movies in the collection by their release year using the bubble sort algorithm.
 - The **findMovieCollectionByReleaseYearRange** method finds and prints movies released within a specified year range. It takes a start year and an end year as input and displays all movies within the range.
 - Finally, the **displayCollection** method prints all the movie information in the collection.
3. A tester program is created to test these classes. Objects of the *Movie* class are created and added to the movieCollection array using the **addMovie** method. The program performs the following operations:
- Search for movies by title and director using the **findMovieByTitle** and **findMovieCollectionByDirector** methods.
 - Find movies released within a specified year range using the **findMovieCollectionByReleaseYearRange** method.
 - Sort the collection of movies by release year using the **sortMovieCollectionByReleaseYear** method and print the result using the **displayCollection** method.

- Remove a movie from the collection using the **removeMovie** method and print a confirmation.
- Print the updated list of movies in the collection using the **displayCollection** method.

Sample Input and Output

1.

```
run:
Searching for movie(s) by director:
Movies directed by 'Christopher Nolan':
- Title: Inception
  Director: Christopher Nolan
  Type: Science Fiction
  Year: 2010

- Title: The Dark Knight
  Director: Christopher Nolan
  Type: Action
  Year: 2008

Searching for a movie by title:
Movie 'Inception' found:
- Title: Inception
  Director: Christopher Nolan
  Type: Science Fiction
  Year: 2010

Movies released between 1990 and 2010:
- Title: Inception
  Director: Christopher Nolan
  Type: Science Fiction
  Year: 2010

- Title: The Shawshank Redemption
  Director: Frank Darabont
  Type: Drama
  Year: 1994
```

- Title: Pulp Fiction
Director: Quentin Tarantino
Type: Crime
Year: 1994

- Title: The Dark Knight
Director: Christopher Nolan
Type: Action
Year: 2008

Sorted movieCollection by release year:

- Title: The Godfather
Director: Francis Ford Coppola
Type: Crime
Year: 1972

- Title: Pulp Fiction
Director: Quentin Tarantino
Type: Crime
Year: 1994

- Title: The Shawshank Redemption
Director: Frank Darabont
Type: Drama
Year: 1994

- Title: The Dark Knight
Director: Christopher Nolan
Type: Action
Year: 2008

- Title: Inception
Director: Christopher Nolan
Type: Science Fiction
Year: 2010

Removed movie: Pulp Fiction

Updated Movie Collection:

- Title: The Godfather
Director: Francis Ford Coppola
Type: Crime
Year: 1972

- Title: The Shawshank Redemption
Director: Frank Darabont
Type: Drama
Year: 1994

- Title: The Dark Knight
Director: Christopher Nolan
Type: Action
Year: 2008

- Title: Inception
Director: Christopher Nolan
Type: Science Fiction
Year: 2010

BUILD SUCCESSFUL (total time: 0 seconds)

2.

```
run:
Searching for movie(s) by director:
Movies directed by 'Quentin Tarantino':
- Title: Pulp Fiction
  Director: Quentin Tarantino
  Type: Crime
  Year: 1994
```

```
Searching for a movie by title:
Movie 'Doctor Strange' found:
No movie found!
```

```
Movies released between 2000 and 2010:
```

```
- Title: Inception
  Director: Christopher Nolan
  Type: Science Fiction
  Year: 2010
```

```
- Title: The Dark Knight
  Director: Christopher Nolan
  Type: Action
  Year: 2008
```

```
Sorted movieCollection by release year:
```

```
- Title: The Godfather
  Director: Francis Ford Coppola
  Type: Crime
  Year: 1972
```

- Title: Pulp Fiction
Director: Quentin Tarantino
Type: Crime
Year: 1994
- Title: The Shawshank Redemption
Director: Frank Darabont
Type: Drama
Year: 1994
- Title: The Dark Knight
Director: Christopher Nolan
Type: Action
Year: 2008
- Title: Inception
Director: Christopher Nolan
Type: Science Fiction
Year: 2010

Removed movie: The Dark Knight

Updated Movie Collection:

- Title: The Godfather
Director: Francis Ford Coppola
Type: Crime
Year: 1972
- Title: Pulp Fiction
Director: Quentin Tarantino
Type: Crime
Year: 1994
- Title: The Shawshank Redemption
Director: Frank Darabont
Type: Drama
Year: 1994
- Title: Inception
Director: Christopher Nolan
Type: Science Fiction
Year: 2010

BUILD SUCCESSFUL (total time: 0 seconds)

3.

```
run:
Searching for movie(s) by director:
Movies directed by 'Steven Spielberg':
No movie found!
```

```
Searching for a movie by title:
Movie 'The Godfather' found:
- Title: The Godfather
  Director: Francis Ford Coppola
  Type: Crime
  Year: 1972
```

Movies released between 1900 and 2000:

```
- Title: Inception
  Director: Christopher Nolan
  Type: Science Fiction
  Year: 2010

- Title: The Shawshank Redemption
  Director: Frank Darabont
  Type: Drama
  Year: 1994

- Title: Pulp Fiction
  Director: Quentin Tarantino
  Type: Crime
  Year: 1994

- Title: The Dark Knight
  Director: Christopher Nolan
  Type: Action
  Year: 2008
```

- Title: The Godfather
Director: Francis Ford Coppola
Type: Crime
Year: 1972

Sorted movieCollection by release year:

- Title: The Godfather
Director: Francis Ford Coppola
Type: Crime
Year: 1972

- Title: Pulp Fiction
Director: Quentin Tarantino
Type: Crime
Year: 1994

- Title: The Shawshank Redemption
Director: Frank Darabont
Type: Drama
Year: 1994

- Title: The Dark Knight
Director: Christopher Nolan
Type: Action
Year: 2008

- Title: Inception
Director: Christopher Nolan
Type: Science Fiction
Year: 2010

Removed movie: Inception

Updated Movie Collection:

- Title: The Godfather
Director: Francis Ford Coppola
Type: Crime
Year: 1972

- Title: Pulp Fiction
Director: Quentin Tarantino
Type: Crime
Year: 1994

- Title: The Shawshank Redemption
Director: Frank Darabont
Type: Drama
Year: 1994

- Title: The Dark Knight
Director: Christopher Nolan
Type: Action
Year: 2008

BUILD SUCCESSFUL (total time: 0 seconds)

Source Code

```
public class Movie {  
  
    private String title, director, type;  
  
    private int year;  
  
  
    public Movie(String title, String director, int year, String type) {  
  
        this.title = title;  
  
        this.director = director;  
  
        this.year = year;  
  
        this.type = type;  
    }  
  
  
    public void setTitle(String title) {  
  
        this.title = title;  
    }  
  
  
    public String getTitle() {  
  
        return title;  
    }  
  
  
    public void setDirector(String director) {  
  
        this.director = director;  
    }  
}
```

```
public String getDirector() {  
    return director;  
}
```

```
public void setYear(int year) {  
    this.year = year;  
}
```

```
public int getYear() {  
    return year;  
}
```

```
public void setType(String type) {  
    this.type = type;  
}
```

```
public String getType() {  
    return type;  
}
```

```
@Override
```

```
public String toString() {
```

```

        return String.format("- Title: %s\n Director: %s\n Type: %s\n Year: %s\n", title, director,
type, year);
    }
}

```

```

public class MovieCollection {

    private Movie[] movieCollection;

    private int numMovie;

    public MovieCollection() {

        movieCollection = new Movie[100];

        numMovie =0;

    }

    public boolean isEmpty() {

        return numMovie == 0;

    }

    public void addMovie(Movie movie) {

        if (numMovie >= 100) {

            System.out.println("The movie collection is full!");

            return;

        }

        movieCollection[numMovie++] = movie;
    }
}

```

```
}
```

```
public void removeMovie(String title) {  
    for(int i = 0; i<= numMovie; i++) {  
        if(movieCollection[i].getTitle().equalsIgnoreCase(title)) {  
            System.out.println("Removed movie: " + movieCollection[i].getTitle());  
            for(int j = i; j<= numMovie-1; j++) {  
                movieCollection[j] = movieCollection[j+1];  
            }  
            movieCollection[numMovie--] = null; // Nullify the last element  
            return;  
        }  
    }  
    System.out.printf("Movie \"%s\" not found!\n", title );  
}
```

```
public void findmovieCollectionByDirector(String director) {  
    boolean found = false;  
    System.out.printf("Movies directed by '%s':\n", director);  
    for(int i = 0; i< numMovie; i++) {  
        if(movieCollection[i].getDirector().equalsIgnoreCase(director)) {  
            System.out.println(movieCollection[i].toString());  
            found = true;  
        }  
    }  
}
```

```

    }

}

if(!found) {

    System.out.println("No movie found!");

}

}

public void findMovieByTitle(String title) {

    System.out.printf("Movie '%s' found:\n", title);

    for(int i = 0; i< numMovie; i++) {

        if(movieCollection[i].getTitle().equalsIgnoreCase(title)) {

            System.out.println(movieCollection[i].toString());

            return;

        }

    }

    System.out.println("No movie found!");

}

public void sortmovieCollectionByReleaseYear() {

    for(int i = 0; i< numMovie; i++) {

```



```
for(int j = 0; j<numMovie-1; j++) {  
  
    if(movieCollection[j].getYear() > movieCollection[j+1].getYear()) {  
  
        Movie temp = movieCollection[j];  
  
        movieCollection[j] = movieCollection[j+1];  
  
        movieCollection[j+1] = temp;  
  
    }  
  
    if(movieCollection[j].getYear() == movieCollection[j+1].getYear()) {  
  
        if(movieCollection[j].getType().charAt(0)>  
movieCollection[j+1].getType().charAt(0)) {  
  
            Movie temp = movieCollection[j];  
  
            movieCollection[j] = movieCollection[j+1];  
  
            movieCollection[j+1] = temp;  
  
        }  
  
    }  
  
}  
  
}
```

```
public void findmovieCollectionByReleaseYearRange(int startYear, int endYear) {  
  
    for(Movie m: movieCollection) {  
  
        if (m!=null) {  
  
            if (m.getYear() >= startYear && m.getYear() <= endYear) {  
  
                System.out.println(m.toString());  

```

```

    }
}
}
}

```

```

public void displayCollection() {
    if(!isEmpty()) {
        for(Movie m: movieCollection) {
            if(m!=null)
                System.out.println(m.toString());
        }
    } else
        System.out.println("The collection is empty");
}
}

```

```

public class TestProgram {

    public static void main(String[] args) {

        MovieCollection movieCollection = new MovieCollection();

        Movie movie1 = new Movie("Inception", "Christopher Nolan", 2010, "Science Fiction");
        Movie movie2 = new Movie("The Shawshank Redemption", "Frank Darabont", 1994,
                                "Drama");
    }
}

```

```
Movie movie3 = new Movie("Pulp Fiction", "Quentin Tarantino", 1994, "Crime");

Movie movie4 = new Movie("The Dark Knight", "Christopher Nolan", 2008, "Action");

Movie movie5 = new Movie("The Godfather", "Francis Ford Coppola", 1972, "Crime");


movieCollection.addMovie(movie1);

movieCollection.addMovie(movie2);

movieCollection.addMovie(movie3);

movieCollection.addMovie(movie4);

movieCollection.addMovie(movie5);


// Sample 1

System.out.println("Searching for movie(s) by director:");

movieCollection.findmovieCollectionByDirector("Christopher Nolan");


System.out.println("\nSearching for a movie by title:");

movieCollection.findMovieByTitle("Inception");


System.out.println("\nMovies released between 1990 and 2010:");

movieCollection.findmovieCollectionByReleaseYearRange(1990, 2010);


System.out.println("\nSorted movieCollection by release year:");

movieCollection.sortmovieCollectionByReleaseYear();
```

```
movieCollection.displayCollection();
```

```
movieCollection.removeMovie(movie3.getTitle());
```

```
System.out.println("\nUpdated Movie Collection:");
```

```
movieCollection.displayCollection();
```

```
// Sample 2
```

```
System.out.println("Searching for movie(s) by director:");
```

```
movieCollection.findmovieCollectionByDirector("Quentin Tarantino");
```

```
System.out.println("\nSearching for a movie by title:");
```

```
movieCollection.findMovieByTitle("Doctor Strange");
```

```
System.out.println("\nMovies released between 2000 and 2010:");
```

```
movieCollection.findmovieCollectionByReleaseYearRange(2000, 2010);
```

```
System.out.println("\nSorted movieCollection by release year:");
```

```
movieCollection.sortmovieCollectionByReleaseYear();
```

```
movieCollection.displayCollection();
```

```
movieCollection.removeMovie(movie4.getTitle());
```

```
System.out.println("\nUpdated Movie Collection:");
```

```

movieCollection.displayCollection();

// Sample 3

System.out.println("Searching for movie(s) by director:");
movieCollection.findmovieCollectionByDirector("Steven Spielberg");

System.out.println("\nSearching for a movie by title:");
movieCollection.findMovieByTitle("The Godfather");

System.out.println("\nMovies released between 1900 and 2000:");
movieCollection.findmovieCollectionByReleaseYearRange(1900, 2010);

System.out.println("\nSorted movieCollection by release year:");
movieCollection.sortmovieCollectionByReleaseYear();
movieCollection.displayCollection();

movieCollection.removeMovie(movie1.getTitle());

System.out.println("\nUpdated Movie Collection:");
movieCollection.displayCollection();

}

}

```


QUESTION 6

Problem

You are tasked to create an order fulfilment system where customers place orders, and the system tracks inventory, processes orders, and generates invoices.

Solution

The system will handle the following tasks:

1. Add product(s) into inventory
2. Remove product(s) from inventory
3. Calculate total payment of the orders
4. Track the customer's inventory
5. Generate invoices for orders

The Java application will contain the following components:

- 1) Product class
 - a) Attributes:
 - i) name
 - ii) Price
 - iii) stock
 - b) Constructor: A constructor that initializes all attributes.
 - c) Accessors: Methods to retrieve the name and price of the product.
 - d) Methods:
 - i) `reduceStock()` : Decreases the stock when a customer orders the product.
 - ii) `toString()` : Override the `toString()` method to return a formatted string that shows name and price of the product
- 2) Order class
 - a) Attributes:
 - i) product
 - b) Constructor: A constructor that initializes all attributes.
 - c) Accessors: Methods to retrieve the name and price of the product.
 - d) Methods:
 - i) `addProduct(Product product)` : Adds a product to the order.

- ii) `removeProduct(Product product)` : Removes a product from the order.
 - iii) `calculateTotal()` : Calculate and return the total of the order based on prices and quantities of the products added.
- 3) Customer class
 - a) Attributes:
 - i) Name
 - ii) Email
 - iii) `shippingAddress`
 - b) Constructor: A constructor that initializes all attributes.
 - c) Accessors: Methods to retrieve the email, name and `shippingAddress` of the customer.
 - d) Methods:
 - i) `toString()` : Override the `toString()` method to return a formatted string representation of the customer's details.
- 4) Inventory Class
 - a) Attributes:
 - i) `productStock` : Stores products and their available quantities
 - b) Constructor: A constructor that initializes all attributes.
 - c) Methods:
 - i) `addProduct(Product product, int qty)` : Adds products to the inventory with their stock count.
 - ii) `isAvailable(Product product, int qty)` : Checks if a specific product and quantity are available in the stock.
 - iii) `updateStock(Product product, int qty)` : Updates the stock after a successful order.
- 5) Invoice Class
 - a) Attributes:
 - i) Order
 - ii) customer
 - b) Constructor: A constructor that initializes all attributes.
 - c) Methods:
 - i) `toString()` : Override the `toString()` method to return a formatted string that shows customer info, products purchased and the total price.

Sample input and output

run:

Power Bank is out of stock.

Invoice for John Doe

Shipping Address: 1234 Main St

Products:

- Laptop: RM1200.0
- Smartphone: RM800.0

Total Amount: \$2000.0

BUILD SUCCESSFUL (total time: 0 seconds)

run:

Laptop is out of stock.

Smartphone is out of stock.

Invoice for John Doe

Shipping Address: 1234 Main St

Products:

- Power Bank: RM100.0

Total Amount: \$100.0

BUILD SUCCESSFUL (total time: 0 seconds)

Source code

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

class Product {
    private String name;
    private double price;
    private int stock;

    //constructor
    Product(String name, double price, int stock){
        this.name = name;
        this.price = price;
        this.stock = stock;
    }

    //accessor
    public String getName(){
        return name;
    }
    public double getPrice(){
        return price;
    }
    public int getStock(){
        return stock;
    }

    //methods
    void reduceStock(int totalpurchase){
        stock = stock - totalpurchase;
    }
}
```

```
}  
}
```

```
class Order {  
    private List<Product> products;  
  
    //constructor  
    Order(){  
        products = new ArrayList<>();  
    }  
    //accessor  
    List<Product> getProduct(){  
        return products;  
    }  
  
    //methods  
    void addProduct(Product product){  
        products.add(product);  
    }  
    void removeProduct(Product product){  
        products.remove(product);  
    }  
    double calculateTotal(){  
        double total = 0;  
        for (Product product : products) {  
            total += product.getPrice();  
        }  
        return total;  
    }  
}
```

```
class Customer {
```

```

private String name;
private String email;
private String shippingAddress;

//constructor
Customer(String name, String email, String shippingAddress){
    this.name = name;
    this.email =email;
    this.shippingAddress = shippingAddress;
}
//accessor
String getName(){
    return name;
}
String getEmail(){
    return email;
}
String getShippingAddress(){
    return shippingAddress;
}
}

```

```

class Inventory {
    private Map<Product, Integer> productStock;

    //constructor
    Inventory(){
        productStock = new HashMap<>();
    }

    void addProduct(Product product, int qty){
        productStock.put(product, qty);
    }
}

```

```

    }

    boolean isAvailable(Product product, int qty){
        //return true if there is product in the map and the quantity is enough
        return productStock.containsKey(product) && (productStock.get(product) >= qty);
    }

    void updateStock(Product product, int qty){
        if(isAvailable(product, qty)){
            productStock.put(product, productStock.get(product) - qty);
        }
    }
}

class Invoice {
    private Order order;
    private Customer customer;
    private double totalAmount;

    //constructor
    Invoice(Order order, Customer customer){
        this.order = order;
        this.customer = customer;
        this.totalAmount = order.calculateTotal();
    }

    // Method to generate the invoice as a string
    @Override
    public String toString() {
        StringBuilder invoice = new StringBuilder();
        invoice.append("Invoice for ").append(customer.getName()).append("\n");
        //invoice.append("Email: ").append(customer.getEmail()).append("\n");
    }
}

```

```

        invoice.append("Shipping Address:
").append(customer.getShippingAddress()).append("\n\n");

        invoice.append("Products:\n");
        //loop inside the list of product
        for (Product product : order.getProduct()) {
            invoice.append("- ")
                .append(product.getName())
                .append(": RM")
                .append(product.getPrice())
                .append("\n");
        }

        invoice.append("\nTotal Amount: $").append(totalAmount).append("\n");
        return invoice.toString();
    }
}

```

```

public class Q6 {
    public static void main(String[] args) {
        Product laptop = new Product("Laptop", 1200.00, 10);
        Product phone = new Product("Smartphone", 800.00, 5);
        Product powerBank = new Product("Power Bank", 100.00, 0);

        Customer customer = new Customer("John Doe", "john@example .com", "1234 Main St");

        Inventory inventory = new Inventory();
        inventory.addProduct(laptop, 10);
        inventory.addProduct(phone, 5);
        inventory.addProduct(powerBank, 0);

        Order order = new Order();
    }
}

```

```

    if (inventory.isAvailable(laptop, 1)) {
        order.addProduct(laptop);
        inventory.updateStock(laptop, 1);
    } else {
        System.out.println(laptop.getName() + " is out of stock.");
    }

    if (inventory.isAvailable(phone, 2)) {
        order.addProduct(phone);
        inventory.updateStock(phone, 2);
    } else {
        System.out.println(phone.getName() + " is out of stock.");
    }

    if (inventory.isAvailable(powerBank, 1)) {
        order.addProduct(powerBank);
        inventory.updateStock(powerBank, 1);
    } else {
        System.out.println(powerBank.getName() + " is out of stock.");
    }

    Invoice invoice = new Invoice(order, customer);
    System.out.println(invoice);
}
}

```